

AlgoViz

DSA Visualizer

1. Overview

AlgoViz is a web application that visually explains how data structures and algorithms work step-by-step.

The application focuses on showing the execution process instead of displaying only the final output.

Users can select an algorithm, provide input, and watch the operations through an interactive visualization.

2. What I Will Create

- A web-based platform for learning and understanding DSA.
- Interactive visualization of algorithm execution.
- Controls for Play, Pause, Next, Previous, Reset, and Speed.
- Visual indication of elements being compared, moved, inserted, or changed.
- Time and space complexity information for each algorithm.
- A backend REST API to execute algorithms and provide visualization steps.

Initial Algorithms

Category	Algorithms
Sorting	Bubble Sort, Selection Sort, Insertion Sort
Searching	Linear Search, Binary Search

Initial Data Structures

- Stack
- Queue
- Linked List

Main objective: make DSA concepts easier to understand through visual interaction.

3. Problem We Solve

Students often find DSA difficult because they see the code and final answer but cannot easily see what happens during execution.

Problems

- Static code does not clearly show every operation.
- Comparisons and swaps are difficult to follow.
- Data structure operations are often taught theoretically.
- Time complexity can feel disconnected from actual execution.
- Students have limited opportunity to interact with an algorithm.

Our Solution

- Display algorithms visually, step-by-step.
- Highlight the elements involved in the current operation.
- Show the current state of the data structure.
- Allow users to move through execution manually or automatically.
- Display complexity information with the visualization.

4. How the Project Works

Layer	Responsibility
React Frontend	User interface, controls, and visual animations
Spring Boot Backend	REST APIs and algorithm execution
Java Algorithm Layer	Implement algorithms and generate execution steps
MySQL Database	Store users, history, and progress

Backend response

- Algorithm name
- Current data state
- Elements involved in the operation
- Operation performed
- Explanation of the current step
- Time and space complexity

The backend sends these details through REST APIs, and the React frontend uses them to create the visualization.

5. Technologies Used

Technology	Purpose
Java 21	Implement algorithms and backend logic
Spring Boot	Build the Java backend application
Spring Web	Create REST APIs
React.js	Build the interactive frontend
JavaScript	Frontend logic and animation control
HTML + CSS	Structure and styling
MySQL	Store application data
Spring Data JPA	Database access using Java objects
Hibernate	ORM used with JPA
Spring Security	Authentication and access control
JWT	Token-based authentication
Maven	Dependency management and build
Postman	REST API testing
Git + GitHub	Version control and project showcase

6. Main Project Modules

- Algorithm Visualization — execute and display algorithms.
- Sorting — visualize sorting operations.
- Searching — visualize search operations.
- Data Structures — visualize Stack, Queue, and Linked List.
- Complexity — display time and space complexity.
- Authentication — user registration and login.
- History — store previous visualizations.
- Progress — track algorithms practiced.

7. Development Approach

- First build the algorithms in Java.
- Create REST APIs using Spring Boot.
- Test APIs using Postman.
- Build the visualization interface using React.
- Connect React with the Spring Boot APIs.
- Add MySQL for user and history data.
- Add Spring Security and JWT authentication.
- Improve the interface and prepare the project for deployment.

8. Future Additions

Future Feature	Purpose
Merge Sort	Visualize divide-and-conquer sorting
Quick Sort	Visualize partition-based sorting
Binary Search Tree	Visualize insertion, search, deletion, and traversal
Heap	Visualize heap operations
Graph Editor	Allow users to create nodes and edges
BFS / DFS	Visualize graph traversal
Dijkstra	Visualize shortest-path calculation
Algorithm Comparison	Compare two algorithms on the same input
Code Visualization	Highlight code related to each execution step
Learning Mode	Add explanations, questions, and challenges
Progress Dashboard	Show learning activity and completed algorithms
Docker & Deployment	Package and publish the application online

9. Project Scope

- Start with sorting and searching algorithms.
- Add basic data structures such as Stack, Queue, and Linked List.
- Provide an interactive visualization interface.
- Use Spring Boot REST APIs for backend communication.
- Use MySQL for persistent application data.
- Add authentication and user-specific history.
- Keep the architecture modular so more algorithms can be added later.

Project Title

AlgoViz — DSA Visualizer